سُبْحَانَكَ لاَ عِلْمَ لَنَا إِلاَّ مَا عَلَّمْتَنَا ۖ إِنَّكَ أَنْتَ الْعَلِيمُ الْحَكِيمُ

Dedications

In the name of Allah, the Most Gracious, the Most Merciful

To our beloved parents,

whose unwavering support, endless sacrifices, and constant prayers
have been the foundation of our success.

To our teachers and mentors,
who guided us with knowledge and wisdom
throughout our academic journey.

To the team at Novation City,
who welcomed us with open arms and provided
invaluable learning opportunities.

To our families and friends,
whose encouragement and understanding
made this achievement possible.

With deep gratitude and appreciation.

Acknowledgments

I would like to express my sincere gratitude to all those who contributed to the success of this professional internship experience, including academic supervisors, industry mentors, and institutional staff whose guidance, feedback, and practical support were essential to achieving the project objectives.

First, I extend my deepest appreciation to my academic supervisor, **Mr. Ahmed Bin Ayed**, for their invaluable guidance and continuous support throughout this project. Their expertise was instrumental in ensuring academic rigor and alignment with university requirements.

I am profoundly grateful to the Novation City team who welcomed me into their innovative ecosystem. Special thanks to my professional supervisor, **Mr. Mohamed Chrifa**, for their professional insights and technical guidance in Industry 4.0 implementation and digital transformation consulting.

My sincere appreciation goes to Novation City competitiveness cluster for entrusting me with developing the **IoT-REAP (Internet of Things - Remote Equipment Access Platform)**. Building this secure industrial remote operations platform—integrating Proxmox VE virtualization, Apache Guacamole remote desktop, MQTT-based IoT robot control, and AI-powered demand forecasting—has been an invaluable experience for my professional growth in full-stack development and industrial systems integration.

I extend gratitude to EPI Digital School's administration and faculty for providing quality education and academic resources essential for understanding enterprise-grade web application development, security architecture, and modern software engineering practices.

Finally, I am grateful to my family and friends for their unwavering support and encouragement throughout this journey.

Contents

List of Figures

List of Tables

Chapter 1

Conceptual Study

Introduction

This chapter presents the conceptual study of the IoT-REAP platform. It describes the adopted software architecture, models the system through use case diagrams and representative sequence and activity diagrams, and presents the class diagram used to guide realization.

1 Adopted Software Architecture

1.1 Architectural Pattern Overview

The Model-View-Controller (MVC) architectural pattern is a foundational design pattern in software development that promotes separation of concerns, maintainability, and scalability [**?**]. MVC divides applications into three interconnected components:

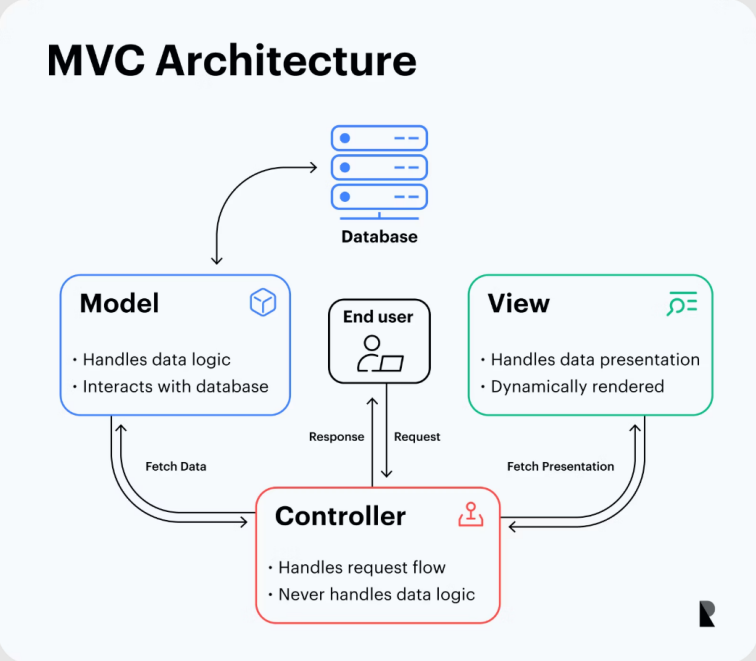

Figure 1.1: MVC Architectural Pattern: Component Interaction Flow

Model Component: Represents data structures, business logic, and data management rules. Responsible for data persistence, retrieval, validation, and enforcing business constraints independently of the user interface.

View Component: Handles presentation layer and user interface rendering. Displays data in appropriate formats and captures user input without containing business logic.

Controller Component: Manages data flow between Models and Views, orchestrating

application logic. Receives user requests, interacts with Models, applies business rules, and selects appropriate Views for rendering responses.

This architectural separation enables independent development, testing, and modification of each component, reducing code coupling and improving maintainability.

1.2 MVC Layers in IoT-REAP Context

The Model Layer encapsulates data structures, database interactions, and business rules. IoT-REAP Models define database structure for users, VM sessions, robots, telemetry, and audit logs. Models implement entity relationships, provide CRUD operations, and enforce data integrity through Eloquent ORM patterns.

The View Layer in IoT-REAP is implemented as a React 18 frontend with TypeScript. Key principles include component-based architecture, typed props interfaces, custom hooks for data fetching, and Zustand for state management. Views render dashboards, session interfaces, robot control panels, and admin consoles.

The Controller Layer orchestrates application logic between Models and Services. IoT-REAP Controllers remain thin—validating input via FormRequests, delegating to Services for business logic, and returning JSON responses via API Resources. Controllers handle authentication/authorization via middleware and manage error responses.

The Service Layer contains all business logic. Services like `VMProvisioningService`, `GuacamoleClient`, `MqttService`, and `RiskScoreEngine` encapsulate complex operations, enabling testability and reuse.

The Repository Layer handles all database queries. Repositories like `VMSessionRepository` and `UserRepository` abstract Eloquent operations, returning Models or Collections—never raw arrays.

Table 1.1: IoT-REAP Platform Core Controllers and Responsibilities

Controller	Primary Responsibilities
AuthController	User authentication, registration, password reset, session management
VMSessionController	Session creation, extension, termination; delegates to VMProvisioningService
VMTemplateController	Template listing, admin CRUD operations for VM templates
ProxmoxNodeController	Node stats, VM listing, VM control (start/stop/reboot)
RobotSessionController	Robot reservation, command dispatch via MqttService
SecurityEventController	Security event listing, active sessions with risk scores
AdminController	User quota management, activity logs, system configuration

1.3 MVC Benefits for IoT-REAP Platform

The MVC architectural pattern provides critical advantages:

Separation of Concerns: Database specialists focus on Model optimization, frontend developers concentrate on React components, backend developers work on Services and Controllers, enabling parallel development without conflicts.

Maintainability: Database schema changes don't require React modifications; UI redesigns don't affect business logic; clear organization enables rapid developer onboarding.

Testability: Models and Services can be unit tested independently; Controllers tested through feature tests with mocked external APIs (Proxmox, Guacamole); React components tested with React Testing Library.

Reusability: Services reused across multiple Controllers (e.g., `ProxmoxClient` used by both `VMSessionController` and `ProxmoxNodeController`); React hooks shared across pages.

Scalability: Independent layer scaling based on performance requirements; AI microservice scales separately from Laravel backend.

Table 1.2: How MVC Architecture Supports IoT-REAP Platform Requirements

Requirement	**MVC Architectural Support**
VM Provisioning	Service layer encapsulates Proxmox API; Controllers orchestrate workflow
Remote Desktop	GuacamoleClient service handles connection lifecycle; Views embed viewer
Robot Control	MqttService publishes commands; WebSocket broadcasts telemetry to Views
AI Forecasting	External microservice called via HTTP; results cached and served to Views
Zero-Trust Security	Middleware layer computes risk scores; Controllers enforce access policies
Audit Logging	Observer pattern on Models; AuditLogger service maintains hash chain
Real-Time Updates	Events broadcast via Laravel Echo; React Views subscribe via WebSocket

The MVC pattern, extended with Service and Repository layers, provides the essential structural foundation enabling the IoT-REAP platform to meet all functional and non-functional requirements while maintaining high code quality and long-term maintainability.

2 Use Case Diagram

After identifying actors and requirements, the conceptual analysis of the platform can be refined through use case modeling. The implemented application suggests one general use case view and several actor-specific refinements. The general model places the platform at the center and connects the four user actors (Guest, Engineer, Teacher, Administrator) as well as the external Stripe actor to the services they invoke: public discovery, learning progression, content authoring, infrastructure access, commerce, and administrative governance.

2.1 General Use Case Diagram

The general use case diagram groups system behavior into four principal subsystems:

- **Public Access and Discovery:** registration, authentication, public browsing, search, certificate verification

- **Learning and Participation:** enrollment, content consumption, quizzes, notes, reviews, forum participation, notifications, certificates
- **Teaching and Publishing:** training path authoring, multimedia management, quiz management, analytics, payout requests, submission for review
- **Remote Laboratory and Administration:** VM provisioning, remote access, reservation management, device control, infrastructure supervision, moderation, refunds, payouts, alerts, and logs

In this model, the Engineer extends the Guest by inheriting public consultation capabilities and adding authenticated learning and virtual laboratory operations. The Teacher extends the Engineer's authenticated access with content authoring and instructional management. The Administrator occupies the supervisory position by controlling both business governance and infrastructure operations. Stripe is modeled as a standalone external actor that interacts with the platform through checkout, webhook callbacks, refunds, and payout-related flows. Figure 1.2 presents the general use case diagram of the IoT-REAP platform, showing the four user actors plus Stripe and their principal interactions with the system subsystems.

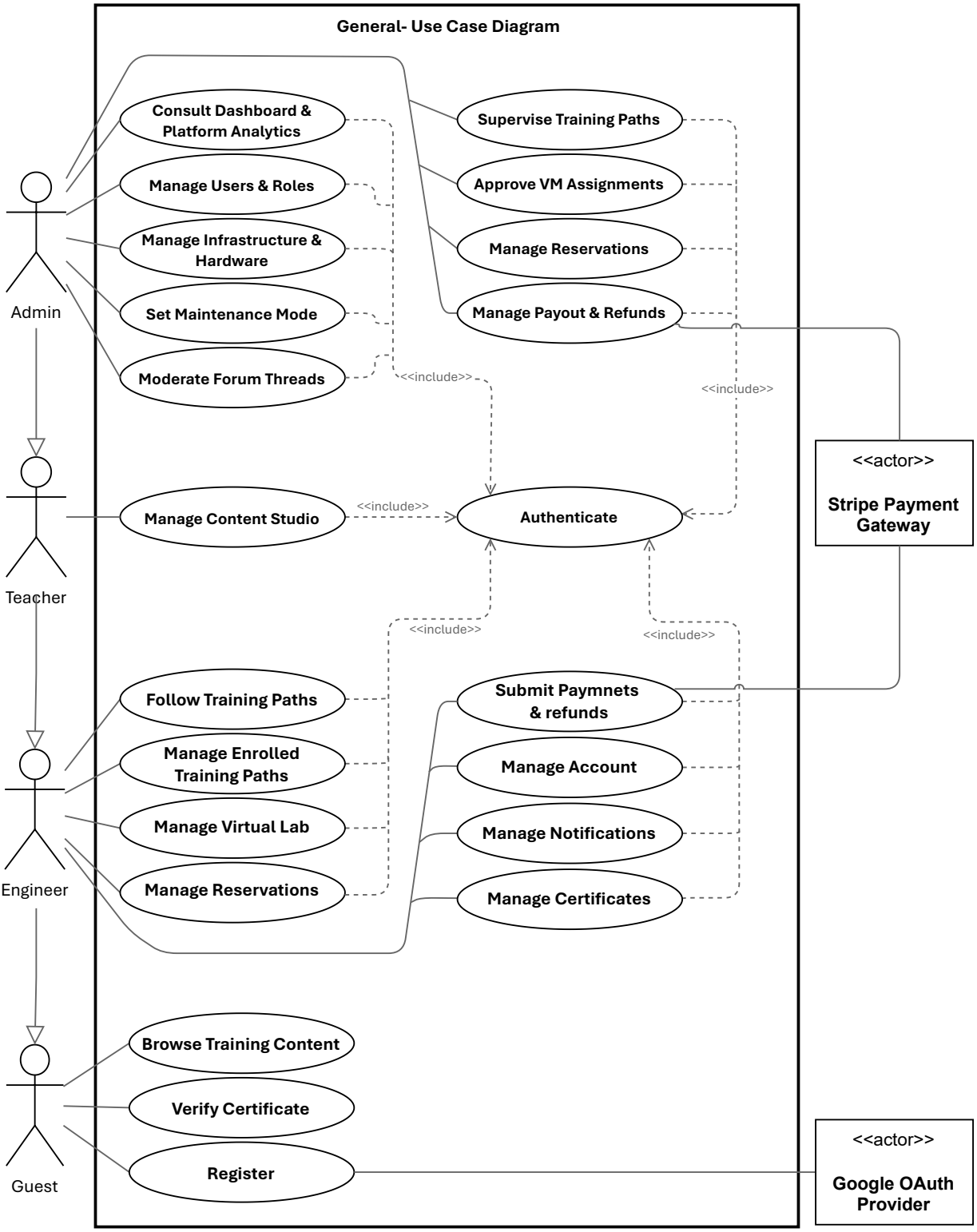

Figure 1.2: General Use Case Diagram

2.2 Refinement and Specification of Use Cases

The major use cases can be refined through representative specifications aligned with the actual implemented routes and domain behaviors. Detailed, actor-specific use case specifications (Guest, Engineer, Teacher, Administrator) are presented in Chapter 3 along with sequence diagrams and concrete implementations; they are not inlined here to avoid duplication between chapters and to keep the conceptual chapter focused on models and high-level diagrams.

2.2.1 Refinement of the Use Case Manage Users & Roles

Figure 1.3 presents the refinement of the Use Case Manage Users & Roles for the Administrator actor. This use case includes the following sub-use cases:

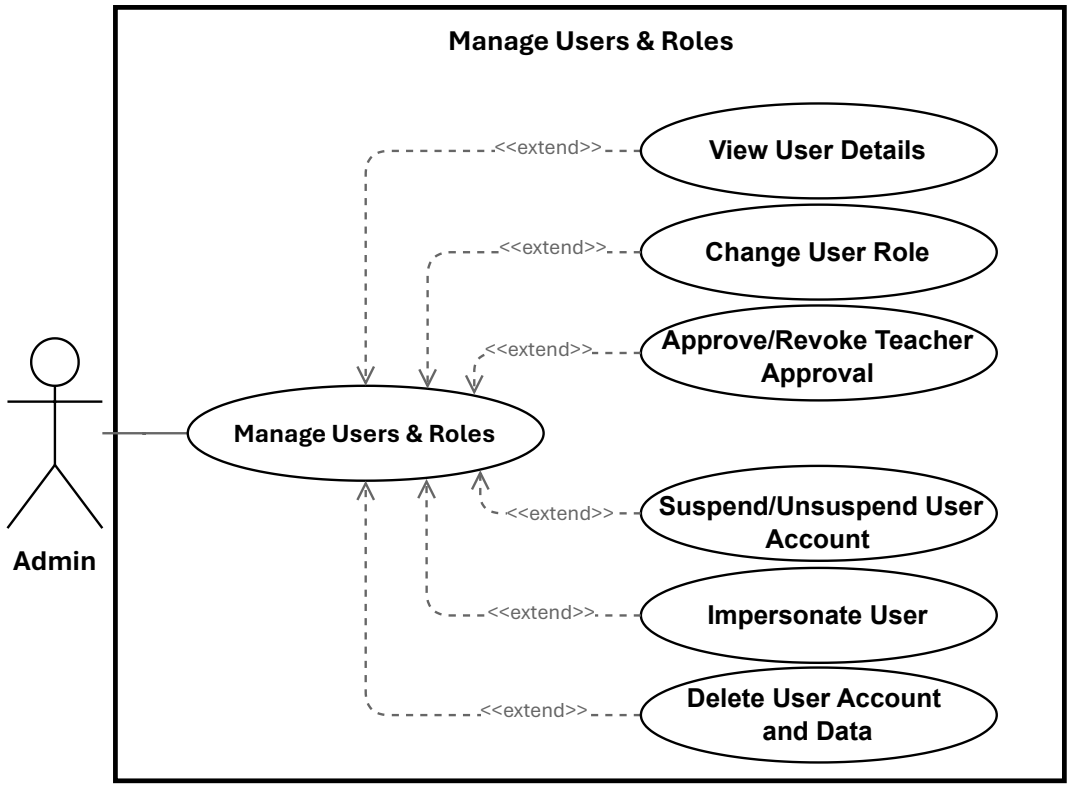

Figure 1.3: Use Case Manage Users & Roles

2.2.1.1 Specification of the Use Case Change User Role

Table 1.3: Use Case Specification: Change User Role

Title	Change User Role
Summary	Administrator updates a user's role (engineer, teacher, admin). Assigning `teacher` also records teacher-approval metadata; assigning a non-teacher clears any teacher-approval metadata.
Actor	Administrator
Precondition	Administrator is authenticated and has admin privileges. Target user exists. Requested role is one of: `engineer`, `teacher`, `admin`.
Post-condition	The user's role is persisted. If role = `teacher`, `teacher_approved_at` and `teacher_approved_by` are set; otherwise those fields are cleared. The change is logged and the updated user is returned.
Basic Scenario	1. Administrator opens the user management list or the target user's detail page. 2. Administrator selects a new role and submits the change. 3. System validates request authorization and that the role value is allowed. 4. System verifies the administrator is not attempting to change their own role. 5. System prepares the update payload (including teacher-approval metadata when applicable). 6. System persists the update to the database. 7. System logs the role change for audit purposes. 8. System returns the updated user record to the admin UI and displays confirmation.
Error Scenario	1. If the administrator attempts to change their own role, the system rejects the request and returns an error. 2. If the submitted role is invalid, the system returns a validation error and no update is made. 3. If persistence or service errors occur, the system aborts the update, logs the failure, and returns an error. 4. If the administrator is not authorized, the system returns an authorization error (403).

2.2.1.2 Specification of the Use Case Approve/Revoke Teacher Approval

Table 1.4: Use Case Specification: Approve/Revoke Teacher Approval

Title	Approve/Revoke Teacher Approval
Summary	Administrator approves a teacher account to mark it as authorized, or revokes an existing teacher approval to remove that authorization. Approving records teacher-approval metadata; revoking clears it.
Actor	Administrator
Precondition	Administrator is authenticated with admin privileges. Target user exists and has the `teacher` role.
Post-condition	If approved, `teacher_approved_at` and `teacher_approved_by` are set. If revoked, those fields are cleared. The change is logged and the updated user record is returned.
Basic Scenario	1. Administrator opens the teacher account details in the user management section. 2. Administrator selects *Approve Teacher* or *Revoke Teacher Approval*. 3. System validates that the target account is a teacher and that the request is authorized. 4. System prepares the update payload for the selected action. 5. System persists the approval change to the database. 6. System logs the action for audit purposes. 7. System returns the updated user record to the admin UI and displays confirmation.
Error Scenario	1. If the target user is not a teacher, the system returns a validation error and no change is made. 2. If the administrator is not authorized, the system returns an authorization error (403). 3. If persistence fails, the system aborts the update, logs the failure, and returns an error.

2.2.1.3 Specification of the Use Case Suspend/Unsuspend User Account

Table 1.5: Use Case Specification: Suspend / Unsuspend User Account

Title	Suspend / Unsuspend User Account
Summary	Administrator temporarily blocks or restores user access. May specify a reason and optional expiration date for temporary suspensions. All actions are recorded for audit.
Actor	Administrator
Precondition	Administrator is authenticated and authorized to manage accounts. Target user exists.
Post-condition	User access status is updated (suspended or active). Metadata (reason, expiration) is recorded. Action is logged for audit. If suspended, user can no longer log in. If unsuspended, access is restored.
Basic Scenario	1. Administrator opens user management and selects an account. 2. Administrator chooses Suspend Account or Restore Access. 3. For suspension: Administrator optionally provides a reason and expiration date. 4. System validates administrator authorization. 5. System checks that target user exists and is eligible (e.g., not an administrator). 6. System updates account status and records metadata. 7. System blocks or restores access immediately. 8. System logs the action for audit. 9. System notifies user of status change (if configured). 10. Administrator receives confirmation and updated status is displayed.
Error Scenario	1. If administrator lacks permission, request is denied and authorization error is displayed. 2. If target user does not exist, system displays error and does not proceed. 3. If target is an administrator, system prevents suspension and displays error. 4. If action cannot be completed due to system errors, system rolls back changes, preserves account state, and displays error. 5. If notifications are configured but unavailable, core action is completed, but system flags notification failure for manual follow-up.

2.2.1.4 Specification of the Use Case Impersonate User

Table 1.6: Use Case Specification: Impersonate User

Title	Impersonate User
Summary	A privileged administrator or support agent temporarily assumes a target user's identity for troubleshooting. Impersonation is time-limited, auditable, and constrained by authorization rules.
Actor	Administrator
Precondition	Initiator is authenticated and authorized to impersonate.
Post-condition	A temporary impersonation session is created. Actions performed are applied as the target user but recorded with initiator metadata. Session start and end are logged.
Basic Scenario	1. Administrator initiates impersonation via UI or API, specifying target user ID and mode (read-only/full). 2. System validates and sanitizes request parameters. 3. System verifies the initiator's permission to impersonate the target. 4. System loads the target user record and verifies eligibility (e.g., active status, lower privilege). 5. System creates a time-limited session/token scoped to the target user, annotated with initiator metadata and mode constraints. 6. System logs an immutable audit event for impersonation start (including session ID, mode, and reason). 7. System returns the token or redirects the initiator into the target's session context; initiator acts within the session and actions execute as the target user but include initiator audit metadata. 8. Initiator ends impersonation, or token expires; system revokes the session/token. 9. System logs impersonation end details (session ID, initiator, target, timestamps).
Error Scenario	1. Unauthorized initiator: System returns HTTP 403 and logs a failed attempt. 2. Target user not found or inactive: System returns HTTP 404/400 and logs failure. 3. Business rule violation (e.g., target has equal/higher privilege): System returns HTTP 403 and logs reason. 4. Session or audit logging failure: Impersonation aborts, returns an error, and clears partial session state.

2.2.1.5 Specification of the Use Case Delete User Account and Data

Table 1.7: Use Case Specification: Delete User Account and Data

Title	Delete User Account and Data
Summary	Administrator removes a user account by anonymizing personal information and removing non-essential data. Payment records preserved for audit.
Actor	Administrator
Precondition	Administrator is authenticated and authorized. Target user exists and is not an administrator. Administrator is not deleting own account.
Post-condition	Account removed. Personal information anonymized. Credentials revoked. Non-essential data removed. Payment history preserved. Deletion audited.
Post-condition (Failure)	No changes made. System reports failure and provides guidance.
Trigger	Administrator confirms deletion via the admin UI.
Basic Scenario	1. Administrator locates target account. 2. Administrator selects user and chooses *Delete Account*. 3. System displays confirmation dialog with deletion details. 4. Administrator confirms deletion. 5. System verifies target is not an administrator and not self. 6. System anonymizes personal information. 7. System removes credentials and session tokens. 8. System removes non-essential data. 9. System preserves financial records. 10. System records deletion in audit logs. 11. System displays success and removes user from active list.
Error Scenario	1. If administrator lacks permission, system rejects request and displays authorization error. 2. If target user is an administrator or administrator attempts to delete own account, system prevents deletion and displays error. 3. If target user does not exist, system displays error and does not proceed. 4. If deletion fails due to system errors, system rolls back changes, preserves user record, and displays error. 5. If partial cleanup fails, system preserves core deletion but flags incomplete cleanup for manual review.

2.2.2 Refinement of the Use Case Manage Infrastructure and Hardware

Figure 1.4 presents the refinement of the Use Case Manage Infrastructure and Hardware for the Administrator actor. This use case includes the following sub-use cases:

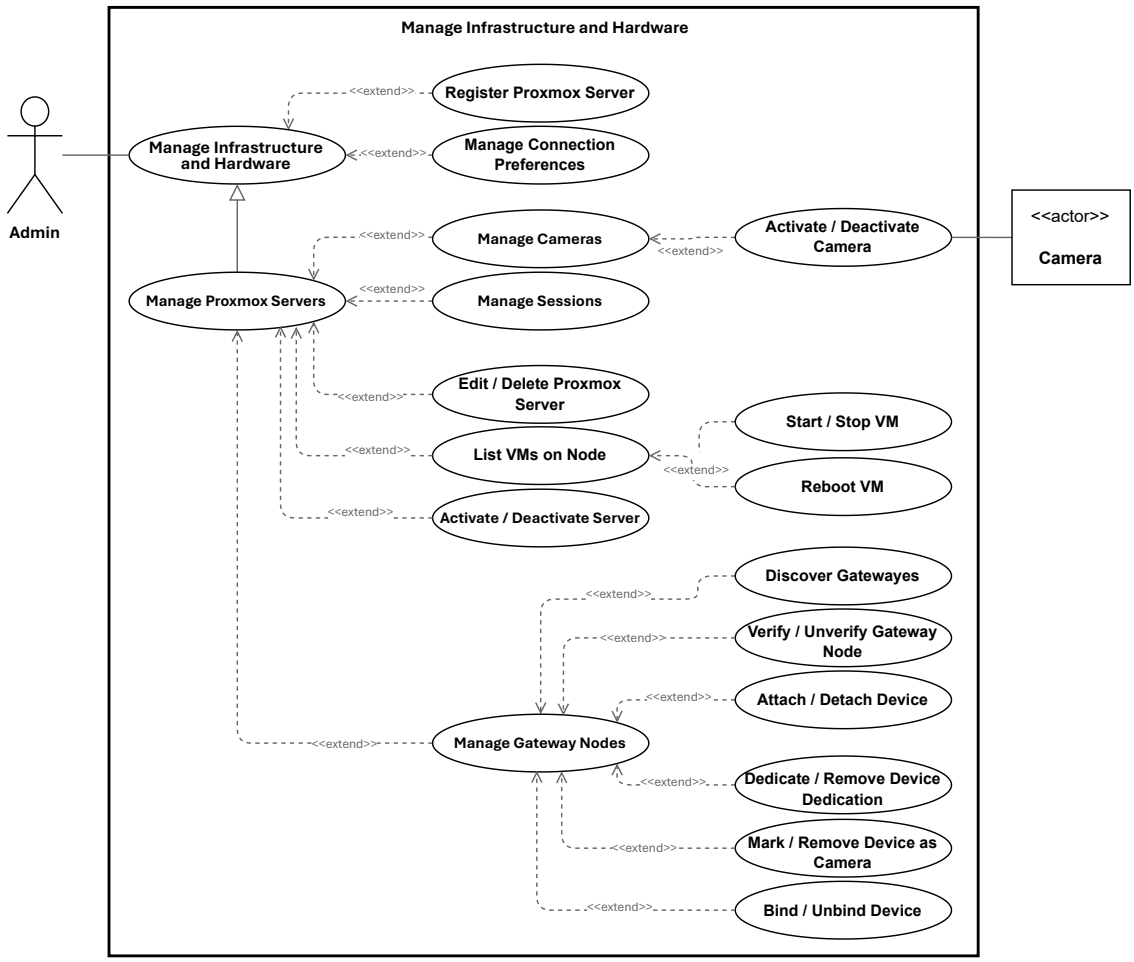

Figure 1.4: Use Case Manage Infrastructure and Hardware

2.2.2.1 Specification of the Use Case Register Proxmox Server

Table 1.8: Use Case Specification: Register Proxmox Server

Title	Register Proxmox Server
Summary	Administrator registers a new Proxmox VE server to enable VM provisioning. System validates connectivity and discovers nodes before adding server to inventory.
Actor	Administrator
Precondition	Administrator is authenticated and authorized. Network access to Proxmox server and valid credentials are available.
Post-condition	Proxmox server is registered and appears in inventory. Discovered nodes are available for provisioning. Configuration is securely stored.
Basic Scenario	1. Administrator opens infrastructure management and selects Register New Server. 2. Administrator enters server details: host address, port, credentials, and optional resource configuration (e.g., VM ID ranges). 3. System validates submitted information. 4. System attempts to connect to Proxmox server using provided credentials. 5. System discovers available cluster nodes. 6. System registers server and stores configuration securely. 7. System displays registered server in inventory with discovered nodes. 8. Administrator can now use this server for provisioning.
Error Scenario	1. If submitted information is invalid or incomplete, system rejects request and displays validation error. 2. If system cannot connect to Proxmox server (unreachable, invalid credentials), system displays error and does not register server. 3. If server is already registered, system informs administrator and suggests updating existing entry. 4. If no cluster nodes are discovered, system flags this and allows administrator to review configuration before proceeding. 5. If system error occurs during registration, system displays error and leaves server unregistered.

2.2.2.2 Specification of the Use Case Edit / Delete Proxmox Server

Table 1.9: Use Case Specification: Edit / Delete Proxmox Server

Title	Edit / Delete Proxmox Server
Summary	Administrator updates the configuration of an existing Proxmox server or removes it from the inventory. Editing updates details like credentials and resource limits; deletion removes the server and its local references from the database.
Actor	Administrator
Precondition	Administrator is authenticated and authorized to manage infrastructure; the target Proxmox server exists in the inventory.
Post-condition	If edited, the updated configuration is persisted and audited. If deleted, the server record and related local references are removed and audited.
Basic Scenario	1. Administrator selects an existing Proxmox server from the inventory. 2. System validates administrator access, loads the server record, and displays management actions. 3. Administrator chooses either *Edit* or *Delete*. 4. **If Edit:** Administrator submits updated details, which the system validates. 5. System tests the connection if host, port, or credentials changed, then persists the new configuration and logs the update. 6. **If Delete:** Administrator confirms deletion. System removes associated local node and session references. 7. System deletes the Proxmox server record and logs the deletion. 8. System returns a success message and refreshes the inventory view.
Error Scenario	1. If submitted edit data is invalid, the system returns a validation error without persisting changes. 2. If the connection test fails during an edit, the system aborts the update and retains the previous configuration. 3. If the target server does not exist, the system returns a not-found error. 4. If deletion fails (e.g., when removing related local records), the system aborts the operation and preserves the existing data.

2.2.2.3 Specification of the Use Case Manage Connection Preferences

Table 1.10: Use Case Specification: Manage Connection Preferences

Title	Manage Connection Preferences
Summary	Administrator creates, updates, tests, or deletes connection preferences (endpoints, ports, protocols, and credentials) for external platform services (Proxmox, Guacamole, MQTT, Gateway). Preferences are stored securely, auditable, and optionally tested for connectivity.
Actor	Administrator
Precondition	Administrator authenticated and authorized; SecretStore available; UI or API accessible; network access to target service if testing is requested.
Post-condition	Preference persisted with secrets stored securely (encrypted); audit record created; test result recorded if requested; dependent processes (e.g., node discovery) triggered when applicable.
Basic Scenario	1. Administrator opens the Connection Preferences editor and submits a create or update request. 2. Controller validates input and forwards to Service. 3. Service encrypts credentials via SecretStore and persists metadata to the repository. 4. If requested, Service invokes an ExternalClient to test connectivity and records the result. 5. Service writes an audit entry (redacting secrets) and returns the persisted preference and test outcome. 6. Controller responds to the Administrator with success and updated resource.
Error Scenario	1. Validation failure: Controller returns HTTP 422; no changes persisted. 2. Unauthorized: Controller returns HTTP 403; attempt logged. 3. SecretStore/repository failure: Service rolls back and returns HTTP 500. 4. Connection test failure: Service records failed test and returns details; persistence may be allowed or rejected per policy.

2.2.2.4 Specification of the Use Case Discover Gateway Nodes

Table 1.11: Use Case Specification: Discover Gateway Nodes

Title	Discover Gateway Nodes
Summary	Administrator initiates a discovery scan to find all available gateway nodes on the registered Proxmox infrastructure and checks their online status.
Actor	Administrator
Precondition	Administrator is authenticated and authorized to manage infrastructure. At least one Proxmox server is registered and reachable.
Post-condition	System identifies all available gateway nodes in the infrastructure. Each discovered node's online/offline status is verified. A list of discovered nodes with their status is displayed to the administrator.
Basic Scenario	1. Administrator opens the infrastructure management interface. 2. Administrator initiates the *Discover Gateway Nodes* operation. 3. System queries all registered Proxmox servers for available gateway nodes. 4. System checks the online status of each discovered node. 5. System displays a list of discovered nodes, including their names, IP addresses, and current status (online/offline). 6. Administrator can review the list and register newly discovered nodes or update existing entries.
Error Scenario	1. If the administrator lacks permission, the request is denied and an authorization error is displayed. 2. If no Proxmox servers are registered, the system informs the administrator and does not attempt discovery. 3. If a Proxmox server is unreachable during discovery, the system skips it and continues with remaining servers. 4. If no gateway nodes are found, the system displays an empty result and suggests verifying the infrastructure setup. 5. If the discovery operation fails due to a system error, the system displays an error message with any partial results found before the failure.

2.2.2.5 Specification of the Use Case Verify / Unverify Gateway Node

Table 1.12: Use Case Specification: Verify / Unverify Gateway Node

Title	Verify / Unverify Gateway Node
Summary	Allows an administrator to mark a gateway node as trusted (verified) or revoke its trusted status (unverified). The system records verification information and may perform a connectivity test before completing the operation.
Actor	Administrator
Precondition	The administrator is authenticated and authorized. The target gateway node exists in the system.
Post-condition	The verification status of the gateway node is successfully updated and an audit record is created.
Basic Scenario	1. The administrator accesses the gateway inventory. 2. The administrator selects a gateway node. 3. The administrator selects the Verify or Unverify option. 4. The system validates the request. 5. The system performs the connectivity test if requested. 6. The system updates the verification status and stores the verification information. 7. The system records the operation in the audit log. 8. The system displays the updated gateway node information.
Error Scenario	1. **Unauthorized Access:** The system rejects the request and informs the administrator. 2. **Validation Error:** The submitted information is invalid and the system requests correction. 3. **Connectivity Test Failure:** The connectivity test fails and the system notifies the administrator. 4. **Gateway Node Not Found:** The selected gateway node does not exist. 5. **System Error:** An unexpected error occurs and the operation is cancelled.

2.2.2.6 Specification of the Use Case Dedicate / Remove Device Dedication

Table 1.13: Use Case Specification: Dedicate / Remove Device Dedication

Title	Dedicate / Remove Device Dedication
Summary	Administrator permanently assigns a USB device to a specific VM for automatic attachment when sessions are created on that VM, or revokes the assignment.
Actor	Administrator
Precondition	Administrator is authenticated and authorized to manage hardware. The USB device exists in the inventory. The target Proxmox server and VM are valid and reachable.
Post-condition	1. (Dedicate) The device is bound to the target VM. Any current attachment or gateway binding is cleared. The device is ready for automatic attachment to future sessions on that VM. Any linked camera is assigned to the VM. 2. (Remove Dedication) The device is no longer bound to any specific VM. Any linked camera assignment is cleared.
Basic Scenario	1. Administrator navigates to the device details in the admin interface. 2. Administrator selects either *Dedicate to VM* or *Remove Dedication*. 3. For dedication: Administrator specifies the target Proxmox server and VM. 4. System validates the request and checks that the device is not actively in use. 5. System applies the dedication (or removes it) and confirms success. 6. System displays the updated device status and binding information.
Error Scenario	1. If the administrator lacks permission, the request is rejected and an authorization error is displayed. 2. If the specified VM or server does not exist, the system rejects the request and asks for valid input. 3. If the device is currently attached to an active session, the system prevents the operation and instructs the user to detach it first. 4. If the operation fails (e.g., due to unavailable services), the system displays an error and leaves the device in its previous state.

2.2.2.7 Specification of the Use Case Activate / Deactivate Camera

Table 1.14: Use Case Specification: Activate / Deactivate Camera

Title	Activate / Deactivate Camera
Summary	Administrator or engineer in an active session activates a robot camera to enable live streaming, or deactivates it to stop the stream.
Actor	Administrator, Engineer (in active session)
Precondition	Actor is authenticated and authorized to control cameras. Target camera exists and is connected to an online robot. For activation, the camera must not already be streaming.
Post-condition	If activated: Camera begins streaming; live video feed is available to authorized users. If deactivated: Camera stops streaming; video feed is no longer available. The action is recorded for audit.
Basic Scenario	1. Actor opens the camera control interface. 2. Actor selects a camera from the available list. 3. Actor chooses either *Start Stream* or *Stop Stream*. 4. System validates the request and checks that the camera and robot are reachable. 5. System sends the command to the robot and waits for confirmation. 6. Robot acknowledges the request and toggles the camera. 7. System verifies the stream is active or stopped as expected. 8. System displays the updated camera status to the actor. 9. If streaming was activated, the live feed is now available in the interface.
Error Scenario	1. If the actor lacks permission to control cameras, the request is denied and an authorization error is displayed. 2. If the camera or robot is not reachable, the system notifies the actor and does not attempt the operation. 3. If the robot does not respond to the command within a reasonable timeout, the system reports the failure and suggests checking robot connectivity. 4. If the camera is already in the requested state (e.g., already streaming when activation is requested), the system displays a notice. 5. If the stream cannot be established after the robot confirms, the system displays an error and reverts the robot camera state if possible.

2.2.2.8 Specification of the Use Case Manage Sessions

Table 1.15: Use Case Specification: Manage Sessions

Title	Manage Sessions
Summary	Administrator manages VM sessions: view, extend, or terminate. Actions are auditable and may trigger cleanup (Guacamole, Proxmox, hardware).
Actor	Administrator (primary). System services: VMSessionController, VMSessionCleanupService, ExtendSessionService, TerminateVMJob, GuacamoleClient, ProxmoxClient, GatewayService, CameraService (secondary).
Precondition	Administrator is authenticated and authorised. Target session exists and is manageable. Guacamole/Proxmox services are reachable.
Post-condition	On success: session expiry is updated (extend) or session is ended and resources are released (terminate). Guacamole connections are removed and devices/camera controls are released. Actions are audited.
Trigger	Administrator opens session management UI or selects Extend/Terminate on a session.
Basic Scenario	1. Administrator opens session view. 2. System expires overdue sessions and lists active sessions. 3. Administrator selects a session and views details. 4. Administrator chooses Extend or Terminate. 5. System validates authorization and session state. 6. If extend: system applies rules, updates `expires_at`, and returns updated session. 7. If terminate: system deletes Guacamole connection, releases camera controls and USB attachments, optionally stops or reverts the VM, and marks session as ended. 8. System refreshes the list and displays confirmation.
Error Scenario	1. If unauthorized, system returns HTTP 403 and logs the attempt. 2. If the session is already ended, system returns no-op or validation error. 3. If extension violates quota or time-window rules, system returns validation error and does not update expiry. 4. If termination cleanup fails (Guacamole/Proxmox/Hardware), system logs the error, retries according to job policy, and returns a warning; the session is force-marked as ended for later reconciliation when safe.

2.2.3 Refinement of the Use Case Manage Content Studio

Figure 1.5 presents the refinement of the Use Case Manage Content Studio for the Teacher actor. This use case includes the following sub-use cases:

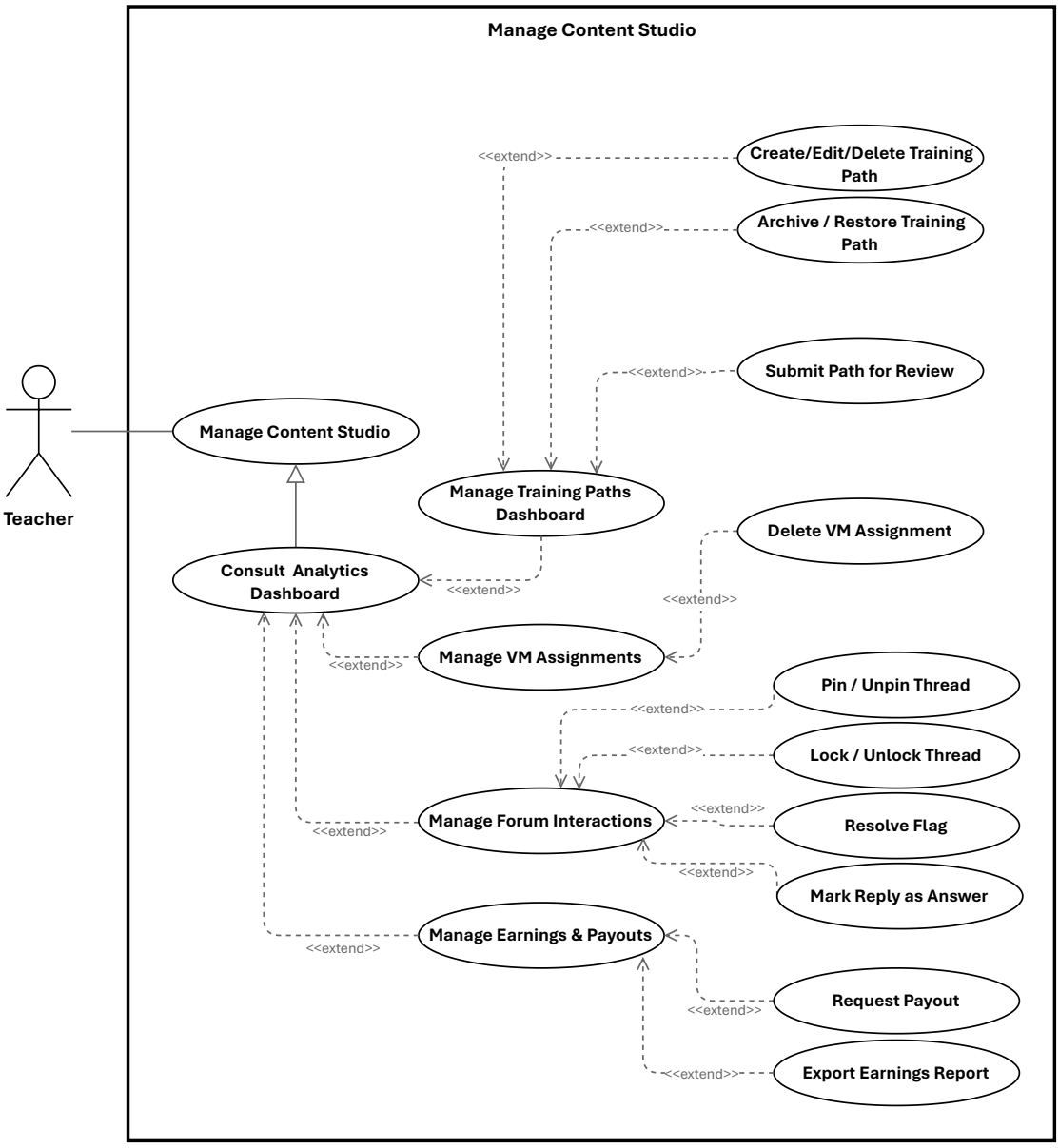

Figure 1.5: Use Case Manage Content Studio

2.2.3.1 Specification of the Use Case Create / Edit / Delete Training Path

Table 1.16: Use Case Specification: Create / Edit / Delete Training Path

Title	Create / Edit / Delete Training Path
Summary	Teacher creates, edits, or removes a training path. New paths start in draft; edits follow ownership rules; deletions require no active enrollments.
Actor	Primary: Teacher. Secondary: Administrator (approval), System.
Precondition	Teacher is authenticated and authorized; categories and VM templates exist; teacher has UI access.
Post-condition	• Create: Training path exists in draft. • Edit: Metadata updated or flagged for approval. • Delete: Training path removed when business rules permit.
Basic Scenario	**A. Create** 1. Teacher provides title, description, category, price, difficulty and modules. 2. System validates permission. 3. System creates a draft training path and may create initial modules. 4. System displays the path for editing. **B. Edit** 1. Teacher submits changes to existing path they own. 2. System validates changes and confirms ownership. 3. If approval required, system marks path pending and notifies admin; otherwise applies changes. 4. System presents updated or pending state. **C. Delete** 1. Teacher requests deletion of an owned path. 2. System verifies ownership and checks for active enrollments. 3. If allowed, system removes the path and content; otherwise informs Teacher why it's blocked. 4. System confirms deletion or shows the blocking reason.
Error Scenario	• Validation failure: system informs Teacher and preserves input. • Unauthorized: system denies action and explains permissions. • Delete blocked: active enrollments prevent deletion; system explains condition. • System error: system preserves state, informs Teacher, and records incident.

2.2.3.2 Specification of the Use Case Archive / Restore Training Path

Table 1.17: Use Case Specification: Archive / Restore Training Path

Title	Archive / Restore Training Path
Summary	Teacher archives a training path to hide it from active listings while preserving content; later restores it to active listings.
Actor	Teacher
Precondition	Teacher is authenticated and owns the training path; it exists in the system.
Post-condition	Archive: path marked archived, removed from active listings, content preserved. Restore: archived flag cleared, path returns to active listings.
Basic Scenario	1. Teacher selects path and chooses "Archive". 2. System asks for confirmation. 3. Teacher confirms. 4. System verifies ownership and archivable state. 5. System marks path as archived, records audit, and removes from active listings. 6. System confirms success; item no longer appears in active lists. 7. Later, Teacher navigates to archived items and chooses "Restore". 8. System asks for confirmation. 9. Teacher confirms. 10. System verifies ownership and archived state, clears archived status, records audit, and restores to active listings. 11. System confirms success; item appears again in active lists.
Error Scenario	1. If Teacher does not own path, system denies action and displays authorization message. 2. If path is not found or already in requested state, system displays validation message. 3. If system persistence error occurs, system preserves consistent state, logs incident, and advises retry. 4. If auxiliary indexing fails, system completes in primary store, records inconsistency for reconciliation, and informs Teacher that listings may update shortly.

2.2.3.3 Specification of the Use Case Delete VM Assignment

Table 1.18: Use Case Specification: Delete VM Assignment

Title	Delete VM Assignment
Summary	Teacher deletes an existing VM assignment they created; the system releases any provisioned VM and remote connections, records the action for audit, and notifies relevant parties.
Actor	Teacher (primary). System (secondary).
Precondition	Teacher is authenticated and authorized to manage the assignment; the VM assignment exists and is eligible for deletion.
Post-condition	On success: the assignment is removed (or soft-deleted per policy); associated external resources are released or scheduled for cleanup; an audit entry is recorded; affected users are notified.
Basic Scenario	1. Teacher locates the VM assignment and chooses Delete. 2. System asks the Teacher to confirm the deletion. 3. Teacher confirms the deletion. 4. System verifies the Teacher's authorization and that the assignment may be deleted. 5. System releases or schedules cleanup of any provisioned VM and removes any remote access/connection resources associated with the assignment. 6. System removes (or marks as deleted) the assignment record and records the deletion in the audit log. 7. System notifies the Teacher and any other relevant parties about the deletion and cleanup status. 8. The UI reflects the deletion and updated resource status.
Error Scenario	1. Assignment not found: system informs the Teacher and makes no changes. 2. Unauthorized (not owner or insufficient privileges): system denies the action and explains required permissions. 3. External resource cleanup fails (e.g., VM removal or remote connection deletion): system logs the failure, retries or enqueues a cleanup job, may perform a soft-delete of the assignment per policy, and notifies the Teacher of the partial outcome and next steps. 4. System error during deletion: system preserves consistent state, records the incident for investigation, and advises the Teacher to retry later or contact support.

2.2.3.4 Specification of the Use Case Pin / Unpin Thread

Table 1.19: Use Case Specification: Pin / Unpin Thread

Title	Pin / Unpin Thread
Summary	Teacher pins or unpins a forum thread within a training path to highlight or remove emphasis. The system updates the thread's visibility and records the moderation action for audit.
Actor	Teacher (primary)
Precondition	Teacher is authenticated and has moderation permission for the training path; the target thread exists and is not deleted.
Post-condition	The thread's pinned state is updated and persisted; forum listings and thread views reflect the change; a moderation audit entry is recorded.
Basic Scenario	1. Teacher opens the forum inbox or thread detail and selects the target thread. 2. Teacher chooses the Pin or Unpin action. 3. System verifies the teacher's authorization and that the thread is in a valid state for moderation. 4. System updates the thread's pinned status and persists the change. 5. System records a moderation audit entry (actor, action, thread identifier, timestamp). 6. System updates forum listings and thread ordering so the change is visible to authorized users. 7. UI displays confirmation and the thread's new pinned state.
Error Scenario	1. Unauthorized: If the teacher lacks moderation permission, the system rejects the action and informs the teacher. 2. Not found / invalid state: If the thread is missing or deleted, the system reports the condition and takes no action. 3. Persistence failure: If the update cannot be saved, the system aborts the change, logs the failure, records an error audit, and notifies the teacher to retry later.

2.2.3.5 Specification of the Use Case Lock / Unlock Thread

Table 1.20: Use Case Specification: Lock / Unlock Thread

Title	Lock / Unlock Thread
Summary	Teacher locks a thread to prevent further replies or unlocks it to allow discussion; the system enforces the state, records a moderation audit entry, and optionally notifies subscribers.
Actor	Teacher (primary)
Precondition	Teacher is authenticated and authorised to moderate the training path containing the thread; the thread exists and is not archived.
Post-condition	On success: the thread's locked state is set (locked/unlocked) and persisted; a moderation audit entry is recorded; subscribers may be notified of the change.
Basic Scenario	1. Teacher selects Lock or Unlock for a target thread and confirms the action. 2. System verifies the teacher's moderation permission and that the thread is in a valid state for the requested action. 3. System updates and persists the thread's locked state. 4. System records a moderation audit entry with actor, action, thread identifier and timestamp. 5. System updates forum listings/permission enforcement so replies are blocked or allowed accordingly. 6. System returns confirmation to the teacher and the UI reflects the updated thread metadata.
Error Scenario	1. Unauthorized: If the teacher lacks moderation permission, the system rejects the action and informs the teacher. 2. Not found / invalid state: If the thread cannot be located or is archived, the system reports the condition and takes no action. 3. Persistence / notification failure: If the locked-state update or notification cannot be saved/sent, the system aborts or rolls back the change, logs the failure, records an error audit, and informs the teacher to retry later.

2.2.3.6 Specification of the Use Case Resolve Flag

Table 1.21: Use Case Specification: Resolve Flag

Title	Resolve Flag
Summary	Teacher reviews a flagged discussion thread within an owned training path, takes an appropriate moderation action, and clears the flag so the thread is no longer treated as requiring attention.
Actor	Teacher
Precondition	Teacher is authenticated and owns the training path containing the flagged thread; the target thread exists and is currently flagged.
Post-condition	The flag is cleared on the target thread and persisted; the thread is removed from flagged-content views; the action is recorded for audit and the teacher receives confirmation.
Basic Scenario	1. Teacher opens the forum inbox or flagged-content view. 2. System displays flagged threads belonging to the teacher's training paths. 3. Teacher selects a flagged thread and reviews the content. 4. Teacher chooses the Resolve Flag action. 5. System verifies ownership and that the thread is still flagged. 6. System clears the flag, persists the updated moderation state, and records an audit entry. 7. System confirms success and the thread is removed from flagged-content views.
Error Scenario	1. Unauthorized: If the teacher does not own the training path, the system rejects the action and keeps the flag unchanged. 2. Invalid state: If the thread is no longer flagged or cannot be found, the system informs the teacher and makes no change. 3. Persistence failure: If the unflag operation cannot be saved, the system preserves the previous state, logs the failure, records an error audit, and prompts the teacher to retry later.

2.2.3.7 Specification of the Use Case Request Payout

Table 1.22: Use Case Specification: Request Payout

Title	Request Payout
Summary	Teacher requests withdrawal of an amount from available earnings. The system validates amount and payment method according to payout policies, creates a pending payout request, and confirms submission.
Actor	Primary: Teacher. Secondary: System.
Precondition	Teacher is authenticated and has earnings data with a non-zero available balance. Earnings/payout interface is available.
Post-condition	Payout request is created with status pending and associated with the teacher account. System confirms successful submission.
Basic Scenario	1. Teacher opens the earnings section and selects Request Payout. 2. System displays payout form with available balance and payment options. 3. Teacher enters payout amount and selects payment method. 4. Teacher submits the request. 5. System validates fields and checks available balance and minimum threshold. 6. System verifies amount and payment method validity. 7. System creates payout request with pending status and confirms submission to the Teacher.
Error Scenario	1. Amount exceeds available balance: system rejects request and displays a validation message; Teacher may retry with a valid amount. 2. Amount below minimum threshold: system rejects request and prompts Teacher to increase amount; Teacher may retry. 3. Invalid payment method: system rejects request and requests a valid method; Teacher may retry. 4. Persistence or service failure: system preserves previous state, logs the incident, records an error audit, and informs the Teacher to retry later.

2.2.3.8 Specification of the Use Case Export Earnings Report

Table 1.23: Use Case Specification: Export Earnings Report

Title	Export Earnings Report
Summary	Teacher exports a CSV report of completed earnings transactions for a selected date range so the income history can be reviewed, reconciled, or archived for accounting purposes. The system generates a downloadable file without changing any financial records.
Actor	Primary: Teacher. Secondary: System.
Precondition	Teacher is authenticated and can access the earnings page. Earnings data is available in the system. If a date range is provided, it is valid.
Post-condition	CSV report is generated for the teacher's completed payments within the selected date range. Browser receives streamed download named earnings-YYYY-MM-DD.csv. No financial data is modified.
Basic Scenario	1. Teacher opens the earnings page and requests an export. 2. System resolves the authenticated teacher and determines the export date range (selected or default). 3. System generates a CSV from the teacher's completed payments for the date range. 4. System streams the CSV file to the teacher's browser and the download starts.
Error Scenario	1. Invalid or empty date range: supplied date range is malformed or inverted; system rejects the request and displays a validation message; no download occurs. 2. Export processing failure: data retrieval or CSV generation fails; system aborts the export, preserves current state, logs the incident, and surfaces a recoverable error. 3. No matching payments: selected date range contains no completed payments; system generates a CSV with header row only and streams it successfully.

2.2.4 Refinement of the Use Case Manage Enrolled Training Paths

Figure 1.6 presents the refinement of the Use Case Manage Enrolled Training Paths for the Engineer actor. This use case includes the following sub-use cases:

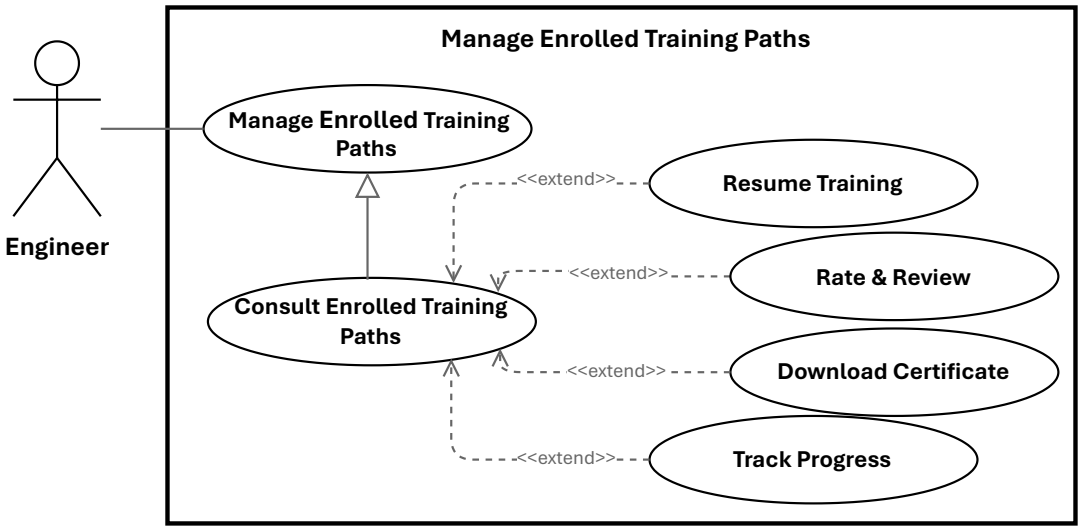

Figure 1.6: Use Case Manage Enrolled Training Paths

2.2.4.1 Specification of the Use Case Resume Training

Table 1.24: Use Case Specification: Resume Training

Title	Resume Training
Summary	Learner resumes a previously started training unit or module at the last saved position so they can continue without losing progress.
Actor	Primary: Engineer (Learner). Secondary: System.
Precondition	Learner is authenticated and enrolled in the training path; the platform stores progress for the unit.
Post-condition	On success: learner resumes at the saved position and the system continues tracking and persisting progress.
Basic Scenario	1. Learner clicks "Resume" on the training unit page. 2. System verifies the learner's authentication status. 3. System verifies that the learner is enrolled and has access to the training unit. 4. System retrieves the last saved progress for the selected unit. 5. System loads the content metadata and associated learning resource. 6. System returns the resume details, including the content location and the starting position. 7. Learner starts playback or continues reading from the saved position. 8. System continues to record progress updates as the learner resumes training.
Error Scenario	1. Authentication failure: session is invalid or expired; the system blocks access and requests re-authentication. 2. Content retrieval failure: the learning resource cannot be loaded; the system displays an error and may allow retry. 3. Progress persistence failure: the system cannot save updates; the previous state is preserved and a recoverable error is shown.

2.2.4.2 Specification of the Use Case Rate & Review

Table 1.25: Use Case Specification: Rate & Review

Title	Rate & Review
Summary	Engineer submits a star rating and optional written review for a training path. The system validates the submission, stores the review, recalculates the training path's average rating, and applies moderation rules when required.
Actor	Engineer
Precondition	Engineer is authenticated and has access to the training path review interface; the target training path exists.
Post-condition	On success: the review is saved or updated, the training path average rating is recalculated, and the appropriate visibility or approval state is set.
Post-condition (Failure)	No review is stored or updated; the previous state is preserved and an error is reported.
Basic Scenario	1. Engineer opens the review form for a training path. 2. Engineer selects a star rating and optionally enters review text, then submits the form. 3. System validates the engineer's eligibility and the submitted input. 4. System saves the review and recalculates the training path's average rating. 5. If an existing review is detected, the system updates the existing record. 6. System confirms the submission, or marks it as pending if moderation is required.
Error Scenario	1. Missing rating: the system rejects the submission with a validation error. 2. Persistence failure: the system aborts the save, preserves the previous state, logs the error, and notifies the engineer. 3. Moderation required: the system stores the review as pending and hides it until approval.

2.2.4.3 Specification of the Use Case Download Certificate

Table 1.26: Use Case Specification: Download Certificate

Title	Download Certificate
Summary	Engineer requests and downloads a completion certificate (PDF) for a training path they have completed. The system verifies eligibility, retrieves or generates the certificate, and returns it as a downloadable file while logging the action.
Actor	Primary: Engineer. Secondary: System.
Precondition	The engineer is authenticated; the engineer has completed the training path and a certificate record exists or can be generated.
Post-condition	The engineer receives the certificate file; a download audit record is created; certificate delivery metadata is recorded.
Basic Scenario	1. Engineer requests a certificate download. 2. System authenticates the request and identifies the engineer. 3. System verifies that the engineer is eligible to download the certificate. 4. System checks whether a certificate file already exists. 5. If a file exists, the system retrieves it; otherwise, the system generates the certificate and stores it. 6. System returns the certificate file or a download link to the engineer. 7. The download starts in the browser or client application. 8. System records the successful download in the audit log.
Error Scenario	1. Unauthenticated request: the system denies access and requests re-authentication. 2. Not eligible: the system rejects the request if the engineer is not entitled to the certificate. 3. Certificate generation or storage error: the system aborts the operation, preserves the current state, and logs the failure.

2.2.5 Refinement of the Use Case Manage Virtual Lab

Figure 1.7 presents the refinement of the Use Case Manage Virtual Lab for the Engineer actor. This use case includes the following sub-use cases:

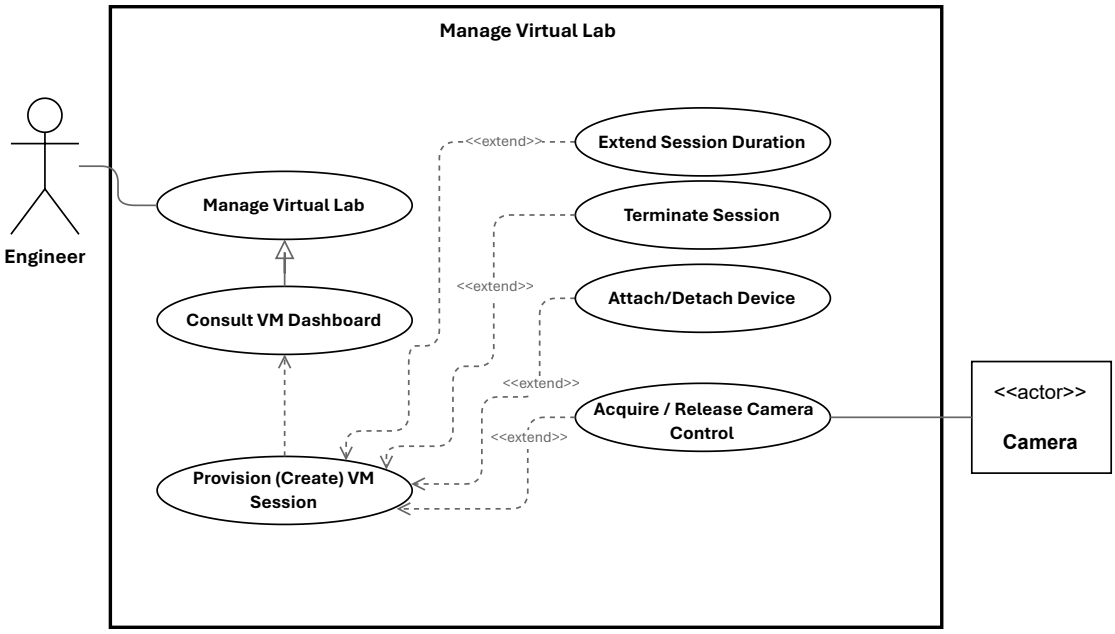

Figure 1.7: Use Case Manage Virtual Lab

2.2.5.1 Specification of the Use Case Provision (Create) VM Session

Table 1.27: Use Case Specification: Provision (Create) VM Session

Title	Provision (Create) VM Session
Summary	Engineer requests an ephemeral VM session by selecting a template and duration. The system validates the request, checks quota, provisions a virtual machine, creates remote access, persists the session, and schedules cleanup.
Actor	Engineer
Precondition	Engineer is authenticated and authorized; the requested template exists and is available; the virtualization and remote-access services are reachable; background processing is operational.
Post-condition	A VM session record exists with status active or provisioning; the VM is provisioned with a host and IP address; remote-access details are created and associated; a cleanup job is scheduled.
Basic Scenario	1. Engineer submits a VM session request with a template and duration. 2. System validates the request and authorizes the engineer. 3. System checks whether the engineer can create a new session. 4. System selects an available node for provisioning. 5. System provisions the virtual machine from the selected template. 6. System waits until provisioning completes and retrieves the VM details. 7. System creates and stores the VM session record. 8. System creates the remote-access connection and associates it with the session. 9. System updates the session status to active and schedules cleanup after the requested duration. 10. System returns the created VM session to the engineer.
Error Scenario	1. Quota exceeded: the system rejects the request and displays an explanatory message. 2. Provisioning failure: the system aborts the operation, attempts cleanup, and marks the session as failed. 3. Persistence failure: the system attempts to roll back the created VM and reports an error. 4. Remote-access setup failure: the system records a partial failure and either schedules remediation or rolls back according to policy.

2.2.5.2 Specification of the Use Case Extend Session Duration

Table 1.28: Use Case Specification: Extend Session Duration

Title	Extend Session Duration
Summary	Engineer requests additional time for an active VM session. The system validates eligibility, checks quota or billing constraints, updates the session expiry, reschedules cleanup, and notifies the engineer.
Actor	Engineer. Secondary: System services.
Precondition	Engineer is authenticated and owns an active VM session; session information is available and the quota or billing service can be consulted.
Post-condition	On success: the session expiry is updated and saved; the cleanup job is rescheduled; quota or billing counters are updated; the engineer is notified and an audit entry is created.
Post-condition (Failure)	No expiry change is made; quota or billing state remains unchanged; the engineer receives an explanatory error and the incident is logged.
Basic Scenario	1. Engineer requests an extension for an active VM session. 2. System authenticates the request and verifies the engineer's ownership of the session. 3. System checks that the session is active and eligible for extension. 4. System verifies quota or billing authorization. 5. If the request is valid, system updates the session expiry, reschedules cleanup, records an audit entry, and notifies the engineer. 6. System returns a success response with the updated session information.
Error Scenario	1. Session not found or not owned: the system rejects the request and returns an authorization or not-found error. 2. Session expired or terminated: the system rejects the request and indicates that a new session must be created. 3. Quota or billing failure: the system rejects the extension request. 4. Persistence or scheduler failure: the system rolls back the update if possible, logs the error, and raises an operational alert.

2.2.5.3 Specification of the Use Case Terminate Session

Table 1.29: Use Case Specification: Terminate Session

Title	Terminate Session
Summary	Engineer ends an active VM session. The system validates authorization, revokes remote access, detaches devices, stops or schedules VM deletion, updates session state, logs the action, and notifies stakeholders.
Actor	Engineer (session owner). Secondary: System.
Precondition	Engineer is authenticated and authorized; the session exists and is in an active state.
Post-condition	On success: the session is marked terminated, remote access is revoked, attached devices are released, the VM is deleted or scheduled for cleanup, an audit entry is recorded, and notifications are sent.
Basic Scenario	1. Engineer issues a terminate request for the session through the UI or API. 2. System authenticates the request and verifies authorization. 3. System marks the session as terminating. 4. System revokes remote access, detaches devices, deletes the VM or schedules cleanup, cancels scheduled jobs, records an audit entry, and notifies stakeholders. 5. System returns a success response with the updated session state.
Error Scenario	1. Authorization failure: the system returns 403 and no change is applied. 2. Persistence failure: the system aborts the operation, logs the error, and returns 500. 3. External API failure: Proxmox or Guacamole errors are retried per policy; persistent failures mark the session with cleanup errors and notify operators.

2.2.5.4 Specification of the Use Case Acquire / Release Camera Control

Table 1.30: Use Case Specification: Acquire / Release Camera Control

Title	Acquire / Release Camera Control
Summary	Engineer acquires exclusive PTZ control of a laboratory camera and later releases it so other users may use the device. The system manages leases, queues, and notifications to coordinate shared access.
Actor	Primary: Engineer. Secondary: Other engineers/users, System Administrator (override), NotificationService, CameraDevice.
Precondition	Engineer is authenticated, has an active session, and the camera is online with PTZ capabilities; control-state tracking is available.
Post-condition	On success: ownership is assigned with a lease expiry; PTZ commands are relayed and acknowledged; on release or lease expiry the camera becomes available and subscribers are notified.
Basic Scenario	1. Engineer requests control of camera C. 2. System verifies authentication, session, and device capabilities. 3. If available, system reserves control and issues a lease with an expiry. 4. System returns success; UI enables PTZ controls and broadcasts a ControlGranted notification. 5. While holding control, the engineer sends PTZ commands which the system relays to the device and acknowledges. 6. Engineer releases control or lease expires; system clears ownership, marks the camera available, and broadcasts ControlReleased.
Error Scenario	1. Camera offline or unreachable: request fails and no ownership change occurs. 2. Unsupported capability: acquire is rejected and PTZ controls remain disabled. 3. Persistence/write failure: system attempts rollback, logs the error, and alerts operations; ownership remains unchanged. 4. Camera busy: system returns BUSY; UI may offer a queue option and the user can re-enter the flow when promoted.

3 Class Diagram

Figure 1.8 presents the general class diagram of our application.

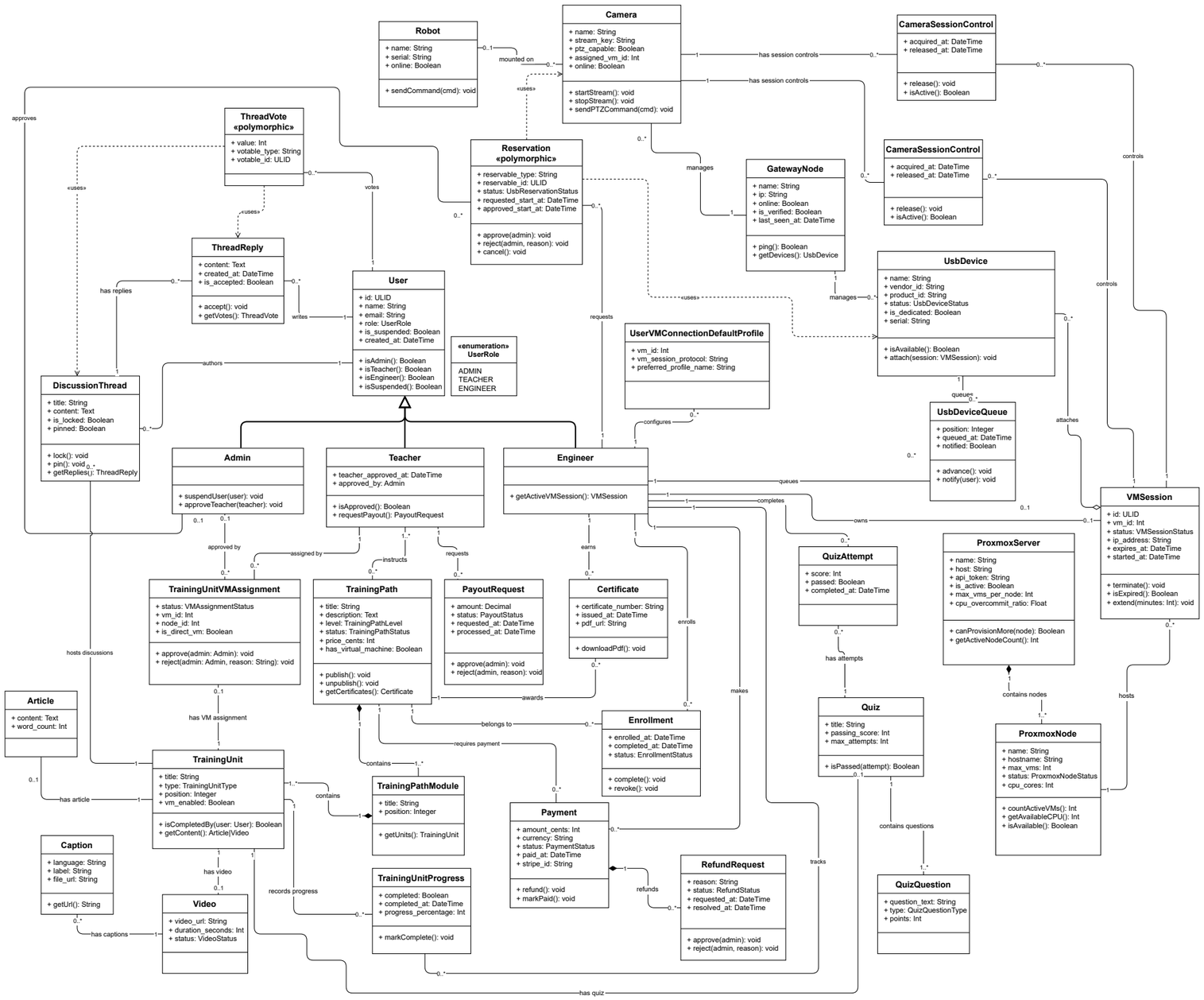

Figure 1.8: Class Diagram of the IoT-REAP Platform

Class User: The class User represents a general platform user and serves as the base entity for all person-like actors. It contains identifying and contact information (such as ULID, name and email), authentication attributes, role membership and suspension flags, and timestamps. User encapsulates shared behaviour and relationships that are common to specialized user types.

Class Admin: The class Admin represents platform administrators with elevated privileges. Admins are responsible for system governance tasks such as user suspension, quota management, teacher approvals, and oversight of infrastructure and financial operations.

Class Teacher: The class Teacher models instructional staff who author and manage training content. Teachers have approval metadata, authoring capabilities, and interfaces for requesting payouts and reviewing learner submissions.

Class Engineer: The class Engineer represents practitioners who consume training materials and operate remote laboratory resources. Engineers interact with VM sessions, hardware reservations, and robot control features.

Class UserRole: An enumeration that defines the possible user roles (ADMIN, TEACHER, ENGINEER). UserRole is used to gate permissions and to differentiate behavior and available features at runtime.

Class TrainingPath: The class TrainingPath models a purchasable or enrollable course consisting of a sequence of modules and units. It stores metadata such as title, description, level, pricing and publication status.

Class TrainingPathModule: TrainingPathModule groups related TrainingUnit objects into a coherent module or chapter within a TrainingPath. Modules provide ordering, summarization and module-level metadata.

Class TrainingUnit: The class TrainingUnit represents an atomic learning unit (for example a lesson or exercise). A unit may be of various types (Article, Video, Quiz) and tracks progress, assignment status and completion criteria.

Class Article: Article is a content subtype representing textual learning material, including body content, attachments and metadata for rendering.

Class Video: Video is a content subtype representing embedded or hosted video media used within a TrainingUnit.

Class Quiz: The Quiz class models assessment units composed of questions. It references QuizQuestion objects and aggregates attempts and scoring rules.

Class QuizQuestion: QuizQuestion represents individual questions within a Quiz, including question text, possible answers and correct answer metadata.

Class QuizAttempt: QuizAttempt records a learner's attempt at a Quiz, including answers chosen, score achieved and timestamps.

Class DiscussionThread: DiscussionThread models a forum-style discussion attached to a TrainingUnit. It aggregates posts and provides moderation and thread metadata.

Class ThreadReply: ThreadReply represents an individual reply within a DiscussionThread; replies may be accepted, voted on, and linked to their authors.

Class Enrollment: Enrollment links a User to a TrainingPath, capturing enrolment date, progress, and entitlement to access path materials and resources.

Class TrainingUnitProgress: TrainingUnitProgress tracks a learner's status within a specific TrainingUnit (e.g., not started, in progress, completed) and records timestamps and completion evidence.

Class Certificate: Certificate models credential issuance after successful completion of a TrainingPath or assessment; it contains recipient data, issuance date and validation metadata.

Class VMSession: VMSession represents a provisioned virtual machine session used by Engineers. It captures VM identifiers, node allocation, status, expiry time and associations to user reservations, Guacamole connections and cleanup jobs.

Class TrainingUnitVMAssignment: This class links TrainingUnits to VM templates or assignments that require a live virtual machine; it stores assignment status, VM identifiers and approval metadata.

Class UserVMConnectionDefaultProfile: UserVMConnectionDefaultProfile captures per-user connection preferences and default profiles for remote desktop or terminal connections to provisioned VMs.

Class ProxmoxServer: ProxmoxServer represents the Proxmox deployment as a logical entity, containing configuration references, API endpoints and a set of ProxmoxNode objects.

Class ProxmoxNode: ProxmoxNode models an individual compute node within the Proxmox cluster; nodes host VM instances and expose resource metrics used by the scheduler and load balancer.

Class GatewayNode: GatewayNode models infrastructure gateways (for example a dedicated VM or host that proxies hardware or network access) and coordinates services such as USB/IP and camera streaming.

Class UsbDevice: UsbDevice models physical USB devices available for reservation and attachment to VMs, tracking device identifiers, availability status and reservation links.

Class Camera: Camera represents camera hardware in the inventory, including streaming endpoints, formats supported and reservation/attachment state.

Class CameraSessionControl: CameraSessionControl encapsulates session-level control for cameras (attach/detach, format negotiation and stream lifecycle) when cameras are assigned to a VM session.

Class Reservation: Reservation is a polymorphic record representing a requested allocation of a reservable resource (USB device, camera, VM slot). It stores requested and approved time windows, status and references to the reservable entity.

Class UsbDeviceQueue: UsbDeviceQueue models the waiting queue for an unavailable USB device, enabling FIFO assignment and notifications when a device becomes available.

Class Robot: Robot models a remote or simulated robot endpoint that can be reserved or controlled from a VM session; it includes identifiers, command interfaces and telemetry endpoints.

Class Payment: Payment records financial transactions (amount, currency, status, user and training path references) and serves as the source for refund and payout workflows.

Class RefundRequest: RefundRequest models a user-initiated refund claim, including reason, linked payment, eligibility checks and administrative review status.

Class PayoutRequest: PayoutRequest represents teacher-initiated requests to withdraw earned funds; it includes payout amount, recipient details and approval state. withdraw earned funds; it includes payout amount, recipient details and approval state.

4 Sequence Diagrams

Conclusion

This chapter defined the conceptual foundation of IoT-REAP through architecture, use case modeling, sequence and activity modeling, and class-level representation. These elements provide the bridge between the preliminary requirements and the realization of the platform.